Scrawl IDE & scrawl_v2 引擎 — 完整使用手册

> **版本**: 0.2.1
> **引擎后端**: scrawl_v2 (Bevy/Rust 原生桥接 + Python 纯逻辑层)
> **IDE 框架**: PySide6 (Qt for Python)

目录

- 第一部分：快速开始
  - 1.1 安装与启动
  - 1.2 创建第一个项目
  - 1.3 添加精灵
  - 1.4 编写代码
  - 1.5 运行游戏
- 第二部分：IDE 界面详解
  - 2.1 主窗口布局
  - 2.2 菜单栏
  - 2.3 工具栏
  - 2.4 场景树（层级面板）
  - 2.5 检查器（属性面板）
  - 2.6 代码编辑器
  - 2.7 场景编辑器
  - 2.8 资源浏览器
  - 2.9 控制台
  - 2.10 AI 聊天
  - 2.11 设置与偏好
- 第三部分：节点系统
  - 3.1 节点类型总览
  - 3.2 2D 节点
  - 3.3 物理节点
  - 3.4 摄像机
  - 3.5 渲染节点
  - 3.6 音频节点
  - 3.7 粒子系统
  - 3.8 路径节点
  - 3.9 TileMap 地图
  - 3.10 UI 节点

    - 3.11 计时器
    - 3.12 导航寻路
- 第四部分：scrawl v2 引擎 API 参考
    - 4.1 Game — 游戏主控
    - 4.2 Scene — 场景
    - 4.3 Sprite — 精灵
    - 4.4 Node / Node2D — 节点基类
    - 4.5 物理系统 API
    - 4.6 Camera2D — 摄像机
    - 4.7 动画系统 API
    - 4.8 TileMap — 瓦片地图
    - 4.9 ParticleEmitter2D — 粒子
    - 4.10 导航与寻路 API
    - 4.11 UI 控件 API
    - 4.12 音频 API
    - 4.13 Timer — 计时器
    - 4.14 StateMachine — 状态机
    - 4.15 Path2D / Line2D — 路径与线段
    - 4.16 Light2D — 灯光
    - 4.17 数学工具
    - 4.18 输入系统
    - 4.19 信号系统
    - 4.20 事件装饰器
    - 4.21 资源管理
- 第五部分：专用编辑器
    - 5.1 TileMap 编辑器
    - 5.2 动画帧编辑器
    - 5.3 路径编辑器
    - 5.4 导航网格编辑器
- 第六部分：代码生成与运行
    - 6.1 代码生成流程
    - 6.2 生成文件结构
    - 6.3 运行与调试
- 第七部分：代码片段库
    - 7.1 装饰器片段
    - 7.2 功能片段分类
- 第八部分：项目文件格式

    - 8.1 .scrawl 文件结构
    - 8.2 项目目录结构
- 第九部分：键盘快捷键
- 第十部分：已知限制与未完成功能
    - 10.1 完成度总览
    - 10.2 引擎层面未完成
    - 10.3 IDE 层面未完成
    - 10.4 架构层面问题

第一部分：快速开始

1.1 安装与启动

系统要求

- Python 3.10+
- PySide6
- （可选）Rust 工具链 + maturin（用于编译原生引擎桥接）

安装依赖

```
pip install -r scrawl_ide/requirements.txt
```

启动 IDE

```
cd scrawl_ide
python main.py
```

编译原生引擎（可选）

```
# 安装 maturin
pip install maturin

# 编译 Rust 原生桥接
maturin develop
```

> ⚠️ **注意**：如果不编译原生桥接，`Game.run()` 将只输出提示信息而不会打开游戏窗口。详见 第十部分。

1.2 创建第一个项目

1. 启动 IDE 后，点击工具栏 🆕 **新建项目** 或使用 `Ctrl+N`
2. 选择项目保存目录，输入项目名称
3. IDE 将自动创建项目目录结构：

```
我的项目/
├── 我的项目.scrawl      ← 项目主文件
├── assets/
|    ├── images/           ← 图片资源
|    └── sounds/           ← 音频资源
└── scripts/               ← 脚本文件
```

4. 默认会创建一个场景 `MainScene` 和一个精灵 `Player`

1.3 添加精灵

有三种方式添加节点到场景：

方式一：菜单栏

菜单 → **场景** → **添加节点**

方式二：场景树右键

在场景树中右键场景名 → **添加节点**

方式三：快速添加

右键场景 → **快速添加** → 选择常用类型（Sprite、PhysicsSprite、Camera2D、Label、Timer）

点击"添加节点"后会打开 **添加节点对话框**，所有 22 种节点类型按分类显示在树形列表中，支持搜索过滤。

1.4 编写代码

1. 在场景树中**双击**精灵 → 代码编辑器自动打开
2. 编写精灵逻辑代码，示例：

```python
class Player(Sprite):
    def __init__(self):
        super().__init__()
        self.name = "Player"
        self.speed = 5

    @as_main
    def main_loop(self):
        """主循环"""
        while True:
            if self.is_key_pressed("right"):
                self.move_right(self.speed)
            if self.is_key_pressed("left"):
                self.move_left(self.speed)
            yield 0
```

3. 代码编辑器支持**代码片段插入**（右键菜单或片段面板），涵盖移动、外观、检测、物理等 60+ 个模板

1.5 运行游戏

- 按 `F5` 或点击工具栏 ▶ **运行**
- 按 `Shift+F5` 或点击 ⏹ **停止**
- 运行输出会显示在底部**控制台**面板中

第二部分：IDE 界面详解

2.1 主窗口布局

```
┌──────────────────────────────────────────────────────────────┐
│  菜单栏: 文件 | 编辑 | 视图 | 场景 | 运行 | 帮助          |
│  工具栏: [新建] [打开] [保存] [运行] [停止]                 |
├──────────┬──────────────────────┬──────────────────────────┤
│          |                      |                          |
|  场景树   |     场景编辑器        |   代码编辑器 / 检查器    |
|  (左上)   |     (中央)            |   (右侧，标签页切换)     |
|          |                      |                          |
|──────────|                      |                          |
|          |                      |                          |
| 资源浏览器|                      |                         |
|  (左下)   |                      |                          |
|          |                      |                          |
├──────────┴──────────────────────┴──────────────────────────┤
|  控制台 / AI 聊天 (底部标签页)                              |
└──────────────────────────────────────────────────────────────┘
```

所有面板均可拖拽、浮动、关闭，通过 **视图** 菜单可重置布局。

2.2 菜单栏

文件菜单

操作	快捷键	说明
新建	`Ctrl+N`	创建新项目
打开	`Ctrl+O`	打开 .scrawl 项目
保存	`Ctrl+S`	保存当前项目
另存为	`Ctrl+Shift+S`	另存到新位置
退出	`Ctrl+Q`	退出 IDE

编辑菜单

操作	快捷键	说明
撤销	`Ctrl+Z`	撤销操作
重做	`Ctrl+Y`	重做操作
剪切	`Ctrl+X`	剪切选中
复制	`Ctrl+C`	复制选中
粘贴	`Ctrl+V`	粘贴内容
设置	`Ctrl+,`	打开设置对话框

视图菜单

- **语言**: 切换界面语言（中文 / English 等）
- **重置布局**: 恢复默认面板布局
- **各面板显示/隐藏开关**

场景菜单

操作	说明
添加节点	打开节点类型选择对话框
添加精灵	快速添加 Sprite 节点
添加场景	添加新场景

运行菜单

操作	快捷键	说明
运行	`F5`	生成代码并运行游戏
停止	`Shift+F5`	终止游戏进程

2.3 工具栏

按钮	功能
🆕	新建项目
📁	打开项目
💾	保存
▶️	运行游戏

按钮	功能
⏹	停止游戏

场景工具栏（场景编辑器上方）：

- 工具切换：选择 / 移动 / 绘制
- 缩放控制：适应窗口 / 100% / 缩放滑块
- 网格开关、网格吸附开关
- 网格大小设置

2.4 场景树（层级面板）

显示项目的场景-节点树形结构：

```
📁 我的项目
├── 🎬 MainScene
│   ├── 🎮 Player
│   ├── 📷 Camera
│   └── 🗺️ GameMap
└── 🎬 MenuScene
    ├── 🏷️ Title
    └── ⭕ StartButton
```

右键菜单

在项目根节点上：

- ➕ 添加场景

在场景上：

- ➕ 添加节点（打开完整节点选择对话框）
- 🎨 快速添加（Sprite、PhysicsSprite、Camera2D、Label、Timer）
- ✏️ 重命名
- 🗑️ 删除场景

在精灵/节点上：

- ✏️ 重命名
- 📋 复制
- 🗑️ 删除

添加节点对话框

树形结构展示所有 11 个分类、22 种节点类型，顶部有搜索框可快速过滤：

分类	包含的节点类型
📁 2D节点	Sprite 🎮、PhysicsSprite ⚡、AnimatedSprite2D 🎞️

分类	包含的节点类型
📁 物理	StaticBody2D 🧱、RigidBody2D 🎱、KinematicBody2D 🏃、Area2D 📦
📁 摄像机	Camera2D 📷
📁 渲染	PointLight2D 💡、DirectionalLight2D 🌞、Line2D 📏
📁 音频	AudioPlayer2D 🔊
📁 粒子	ParticleEmitter2D ✨
📁 路径	Path2D 🛤️、PathFollow2D 📍
📁 地图	TileMap 🗺️
📁 UI	Label 🏷️、Button ⭕、ProgressBar 📊、Panel 🖼️
📁 其他	Timer ⏱️、NavigationAgent2D 🧭

2.5 检查器（属性面板）

根据选中对象显示不同的属性面板：

选中精灵/节点时

身份信息：

- 名称（文本框）
- 类名（文本框，对应代码中的 class 名）
- 节点类型（下拉框，可切换节点基类）

变换：

- X / Y 坐标（-10000 ~ 10000）
- 方向角度（-360° ~ 360°）
- 缩放比例（0.01 ~ 100）

外观：

- 可见性开关
- 造型列表（支持右键删除/重命名、双击编辑代码造型）
- 默认造型选择
- 添加造型按钮（图片文件：PNG/JPG/GIF/BMP）
- 添加代码造型按钮（使用代码绘制）

碰撞：

- 碰撞类型（rect / circle / mask）

物理（物理类型节点自动显示）：

- 重力 X / Y

- 摩擦力（0 ~ 1）
- 弹性（0 ~ 1）

碰撞形状（物理节点专属）：

- 形状（rectangle / circle / capsule）
- 宽度 / 高度 / 半径

类型专属属性（根据节点类型自动显示/隐藏）：

- Camera2D：缩放、平滑、跟随目标
- Light2D：颜色、能量、半径、阴影
- AudioPlayer2D：音频文件、音量、音调、自动播放、循环
- ParticleEmitter2D：数量、生命周期、发射开关、预设
- UI 节点：文本内容、字号、文字颜色、进度值
- Timer：等待时间、单次触发、自动开始
- TileMap：单元格大小
- NavigationAgent2D：最大速度、目标位置

选中场景时

- 场景名称
- 背景颜色（颜色选择器）
- 背景图片（文件选择、预览、删除）

选中项目时

- 游戏标题
- 分辨率（宽×高）
- 全屏模式
- FPS
- 调试模式
- 音效列表管理

2.6 代码编辑器

- 双击场景树中的精灵或场景即可打开代码编辑器
- 支持多标签页，每个精灵/场景对应一个标签
- 支持代码片段插入（详见第七部分）
- 代码保存后自动更新到项目模型

2.7 场景编辑器

中央可视化编辑区域，显示当前场景所有节点的可视化表示：

- 不同节点类型有不同的视觉呈现：

    - Sprite：显示造型图片
    - Camera2D：显示视口矩形
    - Light2D：显示光照范围（辉光效果）
    - AudioPlayer2D：显示音频图标
    - ParticleEmitter2D：显示粒子效果
    - TileMap：显示网格
    - UI 节点：显示控件外观
    - 物理体：显示碰撞形状轮廓
    - 路径：显示路径线段

2.8 资源浏览器

显示项目 `assets/` 目录下的文件结构（图片和音频资源）。

2.9 控制台

显示游戏运行时的输出：

- 标准输出（白色文本）
- 错误输出（红色文本）

2.10 AI 聊天

内置 AI 助手面板，支持：

- 选择 AI 后端（OpenAI 兼容接口）
- 配置 API Key、端点地址、模型
- 对话式编程辅助

2.11 设置与偏好

通过 `Ctrl+,` 或 **编辑** → **设置** 打开：

设置项	默认值	说明
编辑器字体	Cascadia Code	代码编辑器字体
字号	12pt	代码字体大小 (8-32)
Tab 宽度	4	Tab 对应空格数
使用空格	✅	Tab 键插入空格
语言	zh_CN	界面语言
AI API Key	—	OpenAI 兼容 API 密钥
AI 端点	https://api.openai.com/v1	API 接口地址

设置项	默认值	说明
AI 模型	gpt-4o-mini	使用的模型

第三部分：节点系统

3.1 节点类型总览

Scrawl IDE 提供 22 种节点类型，设计参考 Godot Engine：

类型	图标	基类	用途
Sprite	🎮	Sprite	基础 2D 精灵
PhysicsSprite	⚡	PhysicsSprite	带物理的精灵（v1兼容）
AnimatedSprite2D	🎞️	AnimatedSprite2D	帧动画精灵
StaticBody2D	🧱	StaticBody2D	静态碰撞体（地面/墙壁）
RigidBody2D	🎱	RigidBody2D	刚体物理
KinematicBody2D	🏃	KinematicBody2D	角色物理
Area2D	📦	Area2D	触发区域
Camera2D	📷	Camera2D	2D 摄像机
PointLight2D	💡	PointLight2D	点光源
DirectionalLight2D	☀️	DirectionalLight2D	平行光
Line2D	📏	Line2D	线段绘制
AudioPlayer2D	🔊	AudioPlayer2D	空间音频
ParticleEmitter2D	✨	ParticleEmitter2D	粒子发射器
Path2D	🛤️	Path2D	路径定义
PathFollow2D	📍	PathFollow2D	路径跟随
TileMap	🗺️	TileMap	瓦片地图
Label	🏷️	Label	文本标签
Button	🔘	Button	按钮
ProgressBar	📊	ProgressBar	进度条
Panel	🖼️	Panel	面板容器
Timer	⏱️	Timer	计时器

类型	图标	基类	用途
NavigationAgent2D		NavigationAgent2D	导航寻路代理

3.2 2D 节点

Sprite

最基础的 2D 精灵，支持造型切换、移动、旋转、缩放、碰撞检测。

```
class Player(Sprite):
    def __init__(self):
        super().__init__()
        self.name = "Player"
        self.add_costume("idle", "assets/images/player.png")

    @as_main
    def main_loop(self):
        while True:
            if self.is_key_pressed("right"):
                self.move_right(5)
            yield 0
```

PhysicsSprite

带物理属性的精灵（v1 兼容），自动响应重力、摩擦、弹性。

```
class Ball(PhysicsSprite):
    def __init__(self):
        super().__init__()
        self.name = "Ball"
        self.set_gravity(0, 9.8)
        self.set_elasticity(0.8)
        self.set_friction(0.3)
```

AnimatedSprite2D

基于 SpriteFrames 的帧动画精灵。在检查器中可使用 动画帧编辑器 管理动画。

```
class Character(AnimatedSprite2D):
    def __init__(self):
        super().__init__()
        self.name = "Character"
        frames = SpriteFrames()
        frames.add_animation("walk")
        frames.add_frame("walk", "walk_01.png")
        frames.add_frame("walk", "walk_02.png")
        frames.set_animation_speed("walk", 12)
        self.sprite_frames = frames
```

```
    @as_main
    def main_loop(self):
        self.play("walk")
        while True:
            yield 0
```

3.3 物理节点

StaticBody2D

静态碰撞体，不受物理驱动，用于地面、墙壁等不动的物体。

```
class Ground(StaticBody2D):
    def __init__(self):
        super().__init__()
        self.name = "Ground"
```

RigidBody2D

刚体物理，受重力和力的作用自动运动。

```
class Crate(RigidBody2D):
    def __init__(self):
        super().__init__()
        self.name = "Crate"
        self.mass = 2.0
        self.gravity_scale = 1.0
        self.bounce = 0.3

    def push(self):
        self.apply_impulse(Vector2(100, -50))
```

KinematicBody2D

角色控制体，手动控制移动，带碰撞滑动。

```
class Player(KinematicBody2D):
    def __init__(self):
        super().__init__()
        self.name = "Player"
        self.speed = 200.0

    @as_main
    def main_loop(self):
        while True:
            velocity = Input.get_vector("move_left", "move_right", "move_up", "move_down")
            self.move_and_slide(Vector2(velocity.x * self.speed, velocity.y * self.speed))
            yield 0
```

Area2D

检测区域，不参与物理碰撞，用于触发器。

```
class Coin(Area2D):
    def __init__(self):
        super().__init__()
        self.name = "Coin"
        self.monitoring = True
```

3.4 摄像机

Camera2D

2D 摄像机，支持跟随目标、平滑移动、视口限制、画面震动。

```
class GameCamera(Camera2D):
    def __init__(self):
        super().__init__()
        self.name = "GameCamera"
        self.smoothing_enabled = True
        self.smoothing_speed = 5.0
        self.zoom = Vector2(1.5, 1.5)

    def follow_player(self, player):
        self.target = player

    def hit_effect(self):
        self.shake(amount=15, duration=0.3)
```

3.5 渲染节点

PointLight2D / DirectionalLight2D

2D 灯光效果。

```
class Torch(PointLight2D):
    def __init__(self):
        super().__init__()
        self.name = "Torch"
        self.color = (255, 200, 100)
        self.energy = 1.5
        self.range = 300
        self.shadow_enabled = True
```

Line2D

线段绘制。

```
class LaserBeam(Line2D):
    def __init__(self):
        super().__init__()
        self.name = "LaserBeam"
        self.width = 3.0
        self.default_color = (255, 0, 0)
        self.add_point(Vector2(0, 0))
        self.add_point(Vector2(200, 100))
```

3.6 音频节点

AudioPlayer2D

空间音频播放器，音量随距离衰减。

```
class BGMPlayer(AudioPlayer2D):
    def __init__(self):
        super().__init__()
        self.name = "BGMPlayer"
        self.stream = AudioStream("assets/sounds/bgm.mp3")
        self.autoplay = True
        self.volume_db = -5.0
```

也可以使用全局 `AudioManager`：

```
AudioManager.load("jump", "assets/sounds/jump.wav")
AudioManager.play("jump")
AudioManager.play_music("assets/sounds/bgm.mp3", loop=True)
```

3.7 粒子系统

ParticleEmitter2D

2D 粒子发射器，内置 7 种预设效果。

```
class Fire(ParticleEmitter2D):
    def __init__(self):
        super().__init__()
        self.name = "Fire"
        # 使用预设
        self.apply_preset("fire")
        self.emitting = True
```

内置预设：`fire`（火焰）、`smoke`（烟雾）、`explosion`（爆炸）、`rain`（雨）、`snow`（雪）、`sparkle`（闪光）、`trail`（轨迹）

或自定义粒子参数：

```
class CustomParticle(ParticleEmitter2D):
    def __init__(self):
        super().__init__()
        self.amount = 50
        self.lifetime = 2.0
        self.direction = Vector2(0, -1)
        self.spread = 45.0
        self.gravity = Vector2(0, 98)
        self.initial_velocity_min = 50
        self.initial_velocity_max = 100
        self.color_start = Color(1, 1, 0)
        self.color_end = Color(1, 0, 0, 0)
```

3.8 路径节点

Path2D

定义一条 2D 路径（点集合）。在检查器中可使用 路径编辑器 可视化编辑。

```
class PatrolPath(Path2D):
    def __init__(self):
        super().__init__()
        self.name = "PatrolPath"
        self.add_point(Vector2(100, 100))
        self.add_point(Vector2(300, 100))
        self.add_point(Vector2(300, 300))
        self.closed = True  # 闭合路径
```

PathFollow2D

沿 Path2D 自动移动的节点。

```
class Follower(PathFollow2D):
    def __init__(self):
        super().__init__()
        self.name = "Follower"
        self.speed = 100.0
        self.loop = True
        self.rotates = True  # 跟随路径旋转
```

3.9 TileMap 地图

TileMap

瓦片地图系统，支持多层、碰撞检测、坐标转换。在检查器中可使用 TileMap 编辑器 可视化绘制。

```
class GameMap(TileMap):
    def __init__(self):
```

```
        tileset = TileSet(tile_size=32)
        tileset.add_tile(1, "grass", collision=False)
        tileset.add_tile(2, "wall", collision=True)
        super().__init__(tileset)
        self.name = "GameMap"

        # 从二维数组加载
        self.load_from_array([
            [2, 2, 2, 2, 2],
            [2, 1, 1, 1, 2],
            [2, 1, 1, 1, 2],
            [2, 2, 2, 2, 2],
        ])

    def check_wall(self, world_x, world_y):
        """检查世界坐标是否是墙壁"""
        map_pos = self.world_to_map(Vector2(world_x, world_y))
        return self.is_cell_solid(map_pos.x, map_pos.y)
```

也可以从字符串地图加载：

```
map_string = """
#####
#...#
#...#
#####
"""
self.load_from_string(map_string, {"#": 2, ".": 1})
```

3.10 UI 节点

Label

文本标签。

```
class ScoreLabel(Label):
    def __init__(self):
        super().__init__()
        self.name = "ScoreLabel"
        self.text = "分数: 0"
        self.font_size = 24
        self.font_color = (255, 255, 0)
```

Button

可点击按钮。

```python
class StartButton(Button):
    def __init__(self):
        super().__init__()
        self.name = "StartButton"
        self.text = "开始游戏"
        self.font_size = 20

        # 连接按钮信号
        self.pressed.connect(self._on_pressed)

    def _on_pressed(self):
        print("游戏开始！")
```

ProgressBar

进度条。

```python
class HealthBar(ProgressBar):
    def __init__(self):
        super().__init__()
        self.name = "HealthBar"
        self.min_value = 0
        self.max_value = 100
        self.value = 100
        self.show_percentage = True
```

Panel

面板容器。

```python
class UIPanel(Panel):
    def __init__(self):
        super().__init__()
        self.name = "UIPanel"
        self.bg_color = (40, 40, 40, 200)
        self.border_width = 2
        self.corner_radius = 8
```

3.11 计时器

Timer

定时触发事件。

```python
class SpawnTimer(Timer):
    def __init__(self):
        super().__init__()
        self.name = "SpawnTimer"
        self.wait_time = 2.0    # 每 2 秒触发
        self.one_shot = False   # 循环触发
        self.autostart = True   # 自动开始

        self.timeout.connect(self._on_timeout)

    def _on_timeout(self):
        print("生成敌人！")
```

3.12 导航寻路

NavigationAgent2D

A* 寻路代理，需要配合 NavigationGrid 使用。在检查器中可使用 导航网格编辑器 可视化编辑障碍物。

```python
class Enemy(NavigationAgent2D):
    def __init__(self):
        super().__init__()
        self.name = "Enemy"
        self.max_speed = 100.0

        # 设置导航网格
        grid = NavigationGrid(width=20, height=15, cell_size=32)
        grid.set_cell_solid(5, 5, True)   # 设置障碍
        self.set_navigation(grid)

        self.target_reached.connect(self._on_arrived)

    def chase_player(self, player_pos):
        self.target_position = player_pos
        velocity = self.get_navigation_velocity()
        # 使用 velocity 移动实体

    def _on_arrived(self):
        print("到达目标！")
```

第四部分：scrawl_v2 引擎 API 参考

4.1 Game — 游戏主控

```
class Game:
    def __init__(self, width=800, height=600, title="Scrawl Game", fps=60, fullscreen=False)
    def set_scene(self, scene: Scene)        # 设置活动场景
    def add_scene(self, scene: Scene)        # 添加场景（不激活）
    def switch_scene(self, scene_name: str)  # 按名称切换场景
    def run(self, fps=None, debug=False, vsync=True)  # 启动游戏循环
```

4.2 Scene — 场景

```
class Scene:
    def __init__(self, name=None)
    def add_sprite(self, sprite)             # 添加精灵
    def remove_sprite(self, sprite)          # 移除精灵
    def set_background_color(self, r, g, b)  # 背景颜色
    def set_background_image(self, path)     # 背景图片
    def broadcast(self, event)               # 广播消息
```

4.3 Sprite — 精灵

属性

属性	类型	说明
`name`	str	名称
`x`, `y`	float	位置
`pos`	_Vec2Proxy	位置代理 (`pos.x`, `pos.y`)
`direction`	float	方向角度 (0=上, 90=右)
`size`	float	缩放比例
`visible`	bool	是否可见
`color`	tuple	RGBA 颜色
`collision_type`	str	碰撞模式 ("rect"/"circle"/"mask")
`scene`	Scene	所属场景
`is_clones`	bool	是否为克隆体

移动方法

```
sprite.move(steps)                  # 沿当前方向移动
sprite.move_up(steps)               # 向上移动
sprite.move_down(steps)             # 向下移动
sprite.move_left(steps)             # 向左移动
sprite.move_right(steps)            # 向右移动
sprite.go_to(x, y)                  # 移动到坐标
sprite.turn_left(degrees)           # 左转
sprite.turn_right(degrees)          # 右转
sprite.point_towards(x, y)          # 面向坐标
sprite.face_towards(target_name)    # 面向目标精灵
sprite.face_random_direction()      # 随机方向
```

外观方法

```
sprite.add_costume(name, image_or_path)   # 添加造型
sprite.switch_costume(name)                # 切换造型
sprite.next_costume()                      # 下一个造型
sprite.show()                              # 显示
sprite.hide()                              # 隐藏
sprite.say(text, duration=2000)            # 显示文字气泡
sprite.set_text(text, font_size=20, color=(255,255,255))  # 设置文字
sprite.set_size(s)                         # 设置缩放
```

碰撞与物理

```
sprite.set_collision_type(mode)    # 设置碰撞模式
sprite.set_gravity(gx, gy)        # 设置重力
sprite.set_friction(f)             # 设置摩擦力
sprite.set_elasticity(e)           # 设置弹性
```

克隆与删除

```
sprite.clone()                     # 克隆自身
sprite.delete_self()               # 删除自身
```

通信

```
sprite.broadcast(event)            # 广播消息
sprite.play_sound(name)            # 播放音效
```

画笔

```
sprite.pen_down()                  # 落笔
sprite.pen_up()                    # 抬笔
sprite.set_pen_color(r, g, b)     # 画笔颜色
sprite.set_pen_size(size)          # 画笔粗细
```

4.4 Node / Node2D — 节点基类

Node（节点树）

```
node.add_child(child)              # 添加子节点
node.remove_child(child)           # 移除子节点
node.get_child(index)              # 按索引获取子节点
node.get_child_count()             # 子节点数量
node.get_children()                # 所有子节点列表
node.get_parent()                  # 父节点
node.get_node("Path/To/Child")    # 按路径获取节点
node.find_child("name")           # 按名称查找
node.find_children("*")           # 查找多个
node.queue_free()                  # 延迟删除
node.is_inside_tree()              # 是否在场景树中
```

分组系统:

```
node.add_to_group("enemies")      # 加入分组
node.remove_from_group("enemies") # 离开分组
node.is_in_group("enemies")       # 是否在分组中
```

元数据:

```
node.set_meta("key", value)       # 设置元数据
node.get_meta("key", default)     # 获取元数据
node.has_meta("key")              # 是否存在
```

生命周期方法（重写）：

```
def _ready(self):                  # 节点就绪时调用
def _process(self, delta):         # 每帧调用
def _physics_process(self, delta): # 物理帧调用
```

信号:

- `tree_entered` — 进入场景树
- `tree_exited` — 离开场景树
- `ready_signal` — 节点就绪
- `child_entered` — 子节点加入

Node2D（2D 变换）

```
node.position = Vector2(100, 200)  # 本地位置
node.global_position               # 世界位置
node.rotation = 1.57               # 旋转（弧度）
node.rotation_degrees = 90         # 旋转（角度）
node.scale = Vector2(2, 2)         # 缩放
node.z_index = 5                   # 绘制顺序
node.visible = True                # 可见性

node.look_at(Vector2(x, y))       # 面向位置
node.rotate(angle)                 # 旋转增量
node.translate(offset)             # 位移增量
node.to_local(global_point)        # 世界→本地坐标
node.to_global(local_point)        # 本地→世界坐标
```

4.5 物理系统 API

CollisionShape2D

```
shape = CollisionShape2D()
shape.set_rect(width, height)          # 矩形碰撞
shape.set_circle(radius)              # 圆形碰撞
shape.set_capsule(radius, height)     # 胶囊碰撞
shape.set_polygon(points)             # 多边形碰撞
shape.disabled = False                 # 是否禁用
```

RigidBody2D

```
body.mass = 2.0                        # 质量
body.gravity_scale = 1.0              # 重力系数
body.linear_velocity = Vector2(x, y)  # 线速度
body.angular_velocity = 0.5           # 角速度
body.linear_damp = 0.1                # 线性阻尼
body.angular_damp = 0.1               # 角阻尼
body.bounce = 0.5                      # 弹性
body.friction = 0.3                    # 摩擦力
body.freeze = False                    # 是否冻结
body.sleeping = False                  # 是否休眠

body.apply_force(Vector2(fx, fy))     # 施加力
body.apply_impulse(Vector2(ix, iy))   # 施加冲量
body.apply_torque(torque)             # 施加扭矩
```

信号：`body_entered`，`body_exited`

KinematicBody2D

```
body.velocity = Vector2(vx, vy)        # 速度
body.up_direction = Vector2(0, -1)     # "上"方向

body.move_and_slide()                  # 移动+滑动
body.move_and_collide(motion)          # 移动+碰撞
body.is_on_floor()                     # 在地面上？
body.is_on_wall()                      # 撞墙？
body.is_on_ceiling()                   # 撞天花板？
body.get_floor_normal()                # 地面法线
```

Area2D

```
area.monitoring = True
area.get_overlapping_bodies()          # 区域内的物体
area.get_overlapping_areas()           # 区域内的区域
area.has_overlapping_bodies()          # 是否有物体
```

信号：`body_entered`，`body_exited`，`area_entered`，`area_exited`

4.6 Camera2D — 摄像机

```
cam.zoom = Vector2(1.5, 1.5)          # 缩放
cam.target = player_node              # 跟随目标
cam.offset = Vector2(0, -50)          # 偏移
cam.smoothing_enabled = True           # 平滑跟随
cam.smoothing_speed = 5.0             # 平滑速度
cam.current = True                     # 设为活动摄像机

# 视口限制
cam.limit_left = 0
cam.limit_right = 2000
cam.limit_top = 0
cam.limit_bottom = 1000

# 画面震动
cam.shake(amount=10, duration=0.5)

# 坐标转换
world_pos = cam.screen_to_world(screen_pos)
screen_pos = cam.world_to_screen(world_pos)
```

4.7 动画系统 API

SpriteFrames & AnimatedSprite2D

```
frames = SpriteFrames()
frames.add_animation("walk")
frames.add_frame("walk", "frame_01.png")
frames.set_animation_speed("walk", 12)
frames.set_animation_loop("walk", True)

anim_sprite = AnimatedSprite2D()
anim_sprite.sprite_frames = frames
anim_sprite.play("walk")
anim_sprite.stop()
anim_sprite.frame  # 当前帧索引
anim_sprite.speed_scale = 1.5
anim_sprite.flip_h = True
```

信号: `animation_finished`, `frame_changed`, `animation_changed`

AnimationPlayer & Animation

```
anim = Animation()
anim.name = "move"
anim.length = 2.0
anim.loop = True
track = anim.add_track("Player", "position")
track.add_keyframe(0.0, Vector2(0, 0))
track.add_keyframe(1.0, Vector2(200, 0))
track.add_keyframe(2.0, Vector2(0, 0))

player = AnimationPlayer()
player.add_animation("move", anim)
player.play("move")
```

信号: `animation_finished`, `animation_started`, `animation_changed`

Tween

```
tween = Tween()
tween.tween_property(sprite, "x", 300.0, 1.5)  # 1.5秒移动到 x=300
tween.play()
```

信号: `finished`, `step_finished`

4.8 TileMap — 瓦片地图

```
tileset = TileSet(tile_size=32)
tileset.add_tile(1, "grass", collision=False, color=(0, 200, 0))
tileset.add_tile(2, "wall", collision=True, color=(100, 100, 100))
```

```
tilemap = TileMap(tileset)

# 单元格操作
tilemap.set_cell(x, y, tile_id)
tilemap.get_cell(x, y)                  # 返回 tile_id 或 -1
tilemap.erase_cell(x, y)
tilemap.clear()

# 批量加载
tilemap.load_from_array([[1, 1, 2], [1, 1, 2]])
tilemap.load_from_string("#..\n#..\n###", {"#": 2, ".": 1})

# 坐标转换
map_pos = tilemap.world_to_map(Vector2(world_x, world_y))
world_pos = tilemap.map_to_world(map_x, map_y)
center = tilemap.map_to_world_center(map_x, map_y)

# 碰撞查询
tilemap.is_cell_solid(x, y)
tilemap.get_collision_cells()

# 多层支持
tilemap.add_layer("foreground")
tilemap.set_cell(x, y, tile_id, layer=1)
```

信号: `changed`

4.9 ParticleEmitter2D — 粒子

```
emitter = ParticleEmitter2D()
emitter.emitting = True
emitter.amount = 100
emitter.lifetime = 2.0
emitter.one_shot = False

# 发射参数
emitter.direction = Vector2(0, -1)
emitter.spread = 45.0
emitter.gravity = Vector2(0, 98)
emitter.initial_velocity_min = 50
emitter.initial_velocity_max = 100
emitter.angular_velocity_min = -90
emitter.angular_velocity_max = 90

# 缩放曲线
emitter.scale_min = 0.5
emitter.scale_max = 1.5
emitter.scale_curve_min = 0.0   # 结束时缩放
emitter.scale_curve_max = 0.0

# 颜色
```

```
emitter.color_start = Color(1, 1, 0, 1)
emitter.color_end = Color(1, 0, 0, 0)

# 发射形状
emitter.emission_shape = ParticleEmitter2D.EMISSION_SHAPE_SPHERE
emitter.emission_sphere_radius = 50

# 预设
emitter.create_preset("fire")          # 内置7种预设
emitter.restart()                      # 重新播放
particles = emitter.get_particles()    # 获取活动粒子列表
```

信号: `finished`

4.10 导航与寻路 API

NavigationGrid

```
grid = NavigationGrid(width=20, height=15, cell_size=32)
grid.diagonal_movement = True

grid.set_cell_solid(5, 5, True)         # 设置障碍
grid.is_cell_solid(5, 5)               # 查询障碍
grid.set_cell_weight(3, 3, 2.0)        # 设置行走代价
grid.clear()                            # 清除所有障碍

# 坐标转换
grid_pos = grid.world_to_grid(world_pos)
world_pos = grid.grid_to_world(gx, gy)

# A* 寻路
path = grid.find_path(start_pos, end_pos)  # 返回 List[Vector2]
```

NavigationAgent2D

```
agent = NavigationAgent2D()
agent.max_speed = 200.0
agent.set_navigation(grid)

agent.target_position = Vector2(500, 300)   # 设置目标
velocity = agent.get_navigation_velocity()   # 获取建议速度
next_pos = agent.get_next_path_position()   # 下一个路点
path = agent.get_current_navigation_path()  # 完整路径

agent.is_target_reached()                   # 到达目标?
agent.is_target_reachable()                 # 目标可达?
agent.is_navigation_finished()              # 导航完成?
```

信号: `target_reached`, `velocity_computed`, `path_changed`

4.11 UI 控件 API

Control（基类）

```
control.size = Vector2(200, 50)
control.min_size = Vector2(100, 30)
control.tooltip_text = "提示文本"
control.mouse_filter = 0  # 0=停止, 1=传递, 2=忽略
control.grab_focus()
control.has_focus()
control.has_point(Vector2(x, y))
```

信号: `resized`, `focus_entered`, `focus_exited`, `mouse_entered`, `mouse_exited`

Label

```
label = Label()
label.text = "Hello World"
label.font_size = 24
label.font_color = (255, 255, 0)
label.align = "center"         # "left", "center", "right"
label.valign = "center"        # "top", "center", "bottom"
label.autowrap = True
label.outline_size = 2
label.outline_color = (0, 0, 0)
```

Button

```
button = Button()
button.text = "Click Me"
button.disabled = False
button.toggle_mode = False
button.flat = False
button.bg_color = (60, 60, 60)
button.hover_color = (80, 80, 80)
button.pressed_color = (40, 40, 40)
button.corner_radius = 4
```

信号: `pressed`, `toggled(pressed)`, `button_down`, `button_up`

ProgressBar

```
bar = ProgressBar()
bar.min_value = 0
bar.max_value = 100
bar.value = 75
bar.show_percentage = True
bar.fill_mode = 0   # 0=左→右, 1=右→左, 2=上→下, 3=下→上
bar.fill_color = (0, 200, 0)
bar.ratio                       # 只读, 0~1
```

信号: `value_changed(value)`

容器

```
hbox = HBoxContainer()          # 水平排列
hbox.separation = 10

vbox = VBoxContainer()          # 垂直排列
vbox.separation = 10

grid = GridContainer()          # 网格排列
grid.columns = 3
grid.h_separation = 10
grid.v_separation = 10
```

Panel

```
panel = Panel()
panel.bg_color = (30, 30, 30, 200)
panel.border_color = (100, 100, 100)
panel.border_width = 2
panel.corner_radius = 8
```

TextEdit / LineEdit

```
text_edit = TextEdit()
text_edit.text = "多行文本编辑器"
text_edit.editable = True
text_edit.font_size = 14
text_edit.get_line_count()

line_edit = LineEdit()
line_edit.text = ""
line_edit.placeholder_text = "请输入..."
line_edit.secret = True         # 密码模式
line_edit.max_length = 20
```

信号: `text_changed`, `text_submitted`

4.12 音频 API

AudioManager（全局静态）

```
AudioManager.load("jump", "sounds/jump.wav")
AudioManager.play("jump", volume_db=0.0)
AudioManager.play_music("sounds/bgm.mp3", volume_db=-5, loop=True)
AudioManager.stop_music()
AudioManager.set_master_volume(-10.0)
AudioManager.mute("Master")
AudioManager.unmute("Master")
```

AudioPlayer / AudioPlayer2D

```
player = AudioPlayer()
player.stream = AudioStream("sound.wav")
player.volume_db = 0.0
player.pitch_scale = 1.0
player.autoplay = False
player.play()
player.stop()
player.pause()
player.resume()
player.seek(1.5)
player.is_playing()
player.get_playback_position()

# 2D 空间音频（距离衰减）
player2d = AudioPlayer2D()
player2d.max_distance = 500.0
player2d.attenuation = 1.0
```

信号：`finished`

4.13 Timer — 计时器

```
timer = Timer()
timer.wait_time = 1.5          # 等待时间（秒）
timer.one_shot = True          # 即仅触发一次
timer.autostart = False        # 是否自动开始
timer.paused = False           # 暂停

timer.start()                  # 开始（可选 time_sec 参数覆盖 wait_time）
timer.start(3.0)               # 以 3 秒为间隔开始
timer.stop()                   # 停止
timer.is_stopped()             # 是否停止
timer.time_left                # 剩余时间
```

信号：`timeout`

4.14 StateMachine — 状态机

```
# 定义状态
class IdleState(State):
    def enter(self, owner):
        print("进入空闲状态")
    def exit(self, owner):
        print("离开空闲状态")
    def update(self, owner, delta):
        if owner.wants_to_run:
            return  # 交给状态机的条件转移

class RunState(State):
    def enter(self, owner):
        print("进入奔跑状态")
    def update(self, owner, delta):
        owner.move(delta * 200)

# 创建状态机
sm = StateMachine()
sm.add_state("idle", IdleState())
sm.add_state("run", RunState())
sm.add_transition("idle", "run", condition=lambda: owner.wants_to_run)
sm.add_transition("run", "idle", condition=lambda: not owner.wants_to_run)
sm.start("idle")

# 强制切换
sm.transition_to("run")
```

信号: `state_changed(new_state_name)`

4.15 Path2D / Line2D — 路径与线段

Path2D

```
path = Path2D()
path.add_point(Vector2(0, 0))
path.add_point(Vector2(100, 0))
path.add_point(Vector2(100, 100))
path.closed = True

path.get_point_count()                    # 点数量
path.get_total_length()                   # 路径总长度
path.get_point_at_offset(50.0)           # 距起点 50 单位处的坐标
path.set_point(1, Vector2(150, 0))       # 修改点
path.remove_point(2)                      # 删除点
```

PathFollow2D

```
follower = PathFollow2D()
follower.speed = 100.0
follower.loop = True
follower.rotates = True

follower.progress            # 当前距离
follower.progress_ratio      # 当前比例 (0~1)
```

Line2D

```
line = Line2D()
line.add_point(Vector2(0, 0))
line.add_point(Vector2(200, 100))
line.width = 5.0
line.default_color = (255, 0, 0)
line.antialiased = True
line.closed = False
line.joint_mode = 2         # 0=none, 1=sharp, 2=round

line.set_point(0, Vector2(10, 10))
line.remove_point(1)
line.clear_points()
line.get_point_count()
```

4.16 Light2D — 灯光

PointLight2D

```
light = PointLight2D()
light.enabled = True
light.color = (255, 200, 100)
light.energy = 1.5
light.range = 300.0
light.falloff = 1.0
light.offset = Vector2(0, -20)
light.shadow_enabled = True
light.shadow_color = (0, 0, 0, 128)
light.blend_mode = PointLight2D.BLEND_ADD
```

DirectionalLight2D

```
sun = DirectionalLight2D()
sun.color = (255, 255, 240)
sun.max_distance = 2000.0
sun.energy = 0.8
```

LightOccluder2D（阴影遮挡）

```
occluder = LightOccluder2D()
occluder.polygon = [Vector2(0,0), Vector2(100,0), Vector2(100,50), Vector2(0,50)]
```

CanvasModulate（环境光）

```
ambient = CanvasModulate()
ambient.color = (50, 50, 80)   # 暗蓝色环境光
```

4.17 数学工具

Vector2

```
v = Vector2(3, 4)
v.length()                    # 5.0
v.normalized()                # Vector2(0.6, 0.8)
v.dot(other)                  # 点积
v.cross(other)                # 叉积
v.distance_to(other)          # 距离
v.angle()                     # 与X轴夹角（弧度）
v.angle_to(other)             # 两向量夹角
v.rotated(angle)              # 旋转（弧度）
v.lerp(target, 0.5)           # 线性插值
v.move_toward(target, delta)  # 朝目标移动
v.clamped(max_length)         # 限制长度
v.reflect(normal)             # 反射
v.bounce(normal)              # 弹射
v.slide(normal)               # 滑动
v.snapped(Vector2(32, 32))    # 网格对齐

# 常量
Vector2.ZERO                  # (0, 0)
Vector2.ONE                   # (1, 1)
Vector2.UP                    # (0, -1)
Vector2.DOWN                  # (0, 1)
Vector2.LEFT                  # (-1, 0)
Vector2.RIGHT                 # (1, 0)

# 运算符
v1 + v2, v1 - v2, v * 2, v / 3
Vector2.from_angle(1.57)      # 从角度创建单位向量
```

Rect2

```
rect = Rect2(x, y, width, height)
rect.position                    # Vector2
rect.size                        # Vector2
rect.end                         # 右下角 Vector2

rect.has_point(Vector2(x, y)) # 点是否在矩形内
rect.intersects(other_rect)   # 是否相交
rect.merge(other_rect)        # 合并
rect.grow(amount)                # 向外扩展
rect.expand(point)               # 扩展以包含点
```

Color

```
c = Color(1.0, 0.5, 0.0, 1.0)    # RGBA (0~1)
c = Color.from_hex("#FF8000")      # 从十六进制
c = Color.rgb8(255, 128, 0)        # 从 0~255

c.r, c.g, c.b, c.a                # 分量
c.lerp(other, 0.5)                 # 颜色插值
c.darkened(0.3)                    # 变暗
c.lightened(0.3)                   # 变亮

# 预定义颜色
Color.WHITE, Color.BLACK, Color.RED
Color.GREEN, Color.BLUE, Color.YELLOW
Color.CYAN, Color.MAGENTA, Color.TRANSPARENT
```

Transform2D

```
t = Transform2D()
t.xform(point)                   # 变换坐标
t.xform_inv(point)               # 逆变换坐标
```

4.18 输入系统

Input（静态类）

```
Input.is_action_pressed("move_right")      # 动作持续按下？
Input.is_action_just_pressed("jump")       # 动作刚按下？
Input.is_action_just_released("jump")      # 动作刚松开？

Input.get_axis("move_left", "move_right")  # -1.0 ~ 1.0
Input.get_vector("left", "right", "up", "down")  # 规范化方向

Input.get_mouse_position()                  # 鼠标位置
```

InputMap

```
InputMap.load_default_actions()            # 加载默认映射

# 默认映射:
# move_left:  K_LEFT, K_A
# move_right: K_RIGHT, K_D
# move_up:    K_UP, K_W
# move_down:  K_DOWN, K_S
# jump:       K_SPACE
# action:     K_RETURN
# cancel:     K_ESCAPE
# attack:     K_Z, MOUSE_LEFT

# 自定义映射
InputMap.add_action("dash")
InputMap.action_add_event("dash", InputEventKey(key="shift"))
```

4.19 信号系统

```
from scrawl_v2 import Signal

class MyNode(Node2D):
    health_changed = Signal()       # 定义信号
    damage_taken = Signal()

    def take_damage(self, amount):
        self.hp -= amount
        self.health_changed.emit(self.hp)  # 发射信号

# 连接信号
node.health_changed.connect(on_health_changed)

# 一次性连接
node.damage_taken.connect(callback, oneshot=True)

# 断开连接
node.health_changed.disconnect(on_health_changed)
```

4.20 事件装饰器

所有装饰器用于标记方法，由引擎在适当时机调用：

```
class MySprite(Sprite):

    @as_main
    def main_loop(self):
        """主循环 -- 必须用 yield"""
        while True:
            self.move_right(1)
            yield 0
```

```
    @as_clones
    def clone_loop(self):
        """克隆体循环"""
        while True:
            yield 0

    @on_key("space", "down")
    def jump(self):
        """按键事件: state = "down" / "up" / "held" """
        self.move_up(20)

    @on_mouse("click")
    def on_click(self):
        """鼠标事件: action = "click" / "move" / "drag" """
        print("鼠标点击")

    @on_sprite_clicked
    def clicked(self):
        """精灵被点击"""
        self.say("你点了我!")

    @on_sprite_collision("Enemy")
    def hit_enemy(self):
        """与指定精灵碰撞"""
        self.delete_self()

    @on_edge_collision
    def hit_edge(self):
        """碰到边界"""
        self.turn_left(180)

    @on_broadcast("game_over")
    def on_game_over(self):
        """收到广播消息"""
        self.hide()
```

4.21 资源管理

```
from scrawl_v2 import ResourceLoader

ResourceLoader.add_search_path("assets/")
ResourceLoader.exists("images/player.png")

texture = ResourceLoader.load("images/player.png")   # ImageTexture
audio = ResourceLoader.load("sounds/jump.wav")        # AudioResource
font = ResourceLoader.load("fonts/pixel.ttf")         # FontResource
```

第五部分：专用编辑器

IDE 为复杂节点类型提供可视化编辑器，在选中对应类型节点时自动显示在检查器面板中。

5.1 TileMap 编辑器

触发条件：选中 `TileMap` 类型节点

界面组成

1. **Tileset 面板**（左侧）
    - 网格显示所有已定义的 Tile 类型
    - 第一个位置为橡皮擦 (✕)
    - 每个 Tile 显示其颜色，碰撞 Tile 有红色边框
    - 点击选中 Tile 后，会显示青色选中框
2. **Tileset 管理按钮**
    - **添加 Tile**：添加新 Tile 类型
    - **删除 Tile**：删除选中的 Tile
    - **编辑颜色**：修改 Tile 颜色
    - **切换碰撞**：设置 Tile 是否有碰撞
3. **地图画布**（中央）
    - 默认 20×15 网格，24px 格子大小
    - 三种绘制工具：
        - 🖌️ **画笔**：点击/拖拽绘制选中 Tile
        - 🧹 **橡皮**：点击/拖拽擦除
        - 🪣 **填充**：flood fill 填充区域
    - **清除全部**：清空整个地图
4. **网格大小控制**
    - 宽度/高度调节（1~100 格）

数据格式

地图数据以 CSV 格式存储（逗号分隔的 Tile ID，行以换行分隔），保存到 `SpriteModel.tilemap_data`。

5.2 动画帧编辑器

触发条件：选中 `AnimatedSprite2D` 类型节点

界面组成

1. **动画选择**（顶部）
    - 动画名称下拉框
    - 添加动画 / 删除动画 / 重命名动画 按钮
2. **动画参数**
    - FPS（1~120，默认 12）
    - 循环开关
3. **帧列表**（中部）
    - 显示帧缩略图（48×48）
    - 添加帧（支持多选图片文件）
    - 删除帧 / 上移 / 下移按钮
4. **预览区**（底部）
    - 96×96 预览窗口
    - 播放 / 停止按钮
    - 帧计数器显示

数据格式

```
{
    "walk": {
        "frames": ["walk_01.png", "walk_02.png", "walk_03.png"],
        "fps": 12,
        "loop": true
    },
    "idle": {
        "frames": ["idle_01.png"],
        "fps": 8,
        "loop": true
    }
}
```

5.3 路径编辑器

触发条件：选中 `Path2D`、`PathFollow2D` 或 `Line2D` 类型节点

操作方式

- **左键点击空白处**：添加路径点
- **拖拽已有点**：移动路径点
- **右键点击点**：删除路径点

设置

- **闭合路径**：是否连接首尾
- **线宽**：0.5 ~ 20
- **线颜色**：颜色选择器

画布特性

- 32px 网格背景
- 原点十字线标记
- 点编号标注
- 路径线段实时预览

5.4 导航网格编辑器

触发条件：选中 `NavigationAgent2D` 类型节点

操作方式

- **左键点击/拖拽**：绘制或擦除障碍物
- 两种工具模式：
  - 🧱 **障碍物**：将格子标记为不可通行
  - 🧹 **擦除**：将格子标记为可通行

设置

- 网格宽度 / 高度
- 障碍物统计（数量和百分比）

数据格式

CSV 格式，0=可通行，1=障碍物。

第六部分：代码生成与运行

6.1 代码生成流程

按 F5 运行时，IDE 进行以下步骤：

1. **代码生成器**读取 ProjectModel 中的所有场景和精灵
2. 生成一个完整的 Python 脚本文件（临时文件）
3. **GameRunner** 启动一个新的 Python 进程执行该脚本
4. 进程的 stdout/stderr 实时输出到控制台面板

6.2 生成文件结构

```python
# -*- coding: utf-8 -*-
"""Generated by Scrawl IDE - 项目名"""

# ======== 导入 ========
from scrawl_v2 import (
    Game, Scene, Sprite, PhysicsSprite,
    on_key, on_mouse, as_main, as_clones,
    # ... 所有节点类型
)

PROJECT_DIR = r"项目路径"

# ======== 游戏配置 ========
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600
GAME_TITLE = "My Game"
GAME_FPS = 60
FULLSCREEN = False

# ======== 精灵类定义 ========
class Player(Sprite):
    def __init__(self):
        super().__init__()
        self.add_costume("idle", r"assets/images/player.png")
        self.name = "Player"
        # ... 用户编写的代码

# ======== 场景类定义 ========
class MainScene(Scene):
    def __init__(self):
        super().__init__()
        self.name = "MainScene"
        self.set_background_color(100, 150, 200)
        player = Player()
        player.go_to(400.0, 300.0)
        self.add_sprite(player)

# ======== 游戏入口 ========
if __name__ == "__main__":
    InputMap.load_default_actions()
    game = Game(width=SCREEN_WIDTH, height=SCREEN_HEIGHT, title=GAME_TITLE)
    game.set_scene(MainScene())
    game.run(fps=GAME_FPS, debug=False)
```

代码注入规则

代码生成器会自动注入以下内容到精灵的 `__init__` 方法中：

- **造型加载代码**：根据 IDE 中添加的造型列表
- **物理属性**：重力、摩擦力、弹性、质量、阻尼
- **碰撞形状**：形状类型和尺寸
- **Camera2D 属性**：缩放、平滑、跟随目标、视口限制
- **Light2D 属性**：颜色、能量、半径、阴影
- **AudioPlayer2D 属性**：音频流、音量、音调、自动播放
- **ParticleEmitter2D 属性**：数量、生命周期、预设
- **UI 属性**：文本、字号、颜色、进度值
- **Timer 属性**：等待时间、单次/循环、自动开始
- **TileMap 属性**：单元格大小
- **节点名称**：`self.name = "节点名"`

6.3 运行与调试

运行环境

- **Python 解释器**：使用当前系统 Python
- **PYTHONPATH**：自动包含 scrawl_v2 模块路径
- **环境变量**：自动设置 UTF-8 编码
- **调试模式**：在游戏设置中启用后，可显示调试覆盖信息

停止游戏

- `Shift+F5` 停止
- 先尝试正常终止（3 秒超时），超时后强制终止

第七部分：代码片段库

代码编辑器内置丰富的代码片段，可通过右键菜单或片段面板插入。

7.1 装饰器片段

片段	说明
`@on_key(key, state)`	键盘事件处理
`@on_mouse(action)`	鼠标事件处理
`@on_sprite_clicked`	精灵点击事件
`@on_sprite_collision(name)`	精灵碰撞事件

片段	说明
`@on_edge_collision`	边缘碰撞事件
`@on_broadcast(message)`	广播消息事件
`@as_main`	主循环（生成器函数）
`@as_clones`	克隆体循环

7.2 功能片段分类

分类	示例片段
移动	`go_to()`, `move()`, `glide_to()`, `point_towards()`, `turn_left()`, `bounce_on_edge()`
外观	`add_costume()`, `switch_costume()`, `say()`, `set_size()`, `show()/hide()`
侦测	`is_touching()`, `is_key_pressed()`, `Input.get_vector()`, `distance_to()`
控制	`clone()`, `delete_self()`, `broadcast()`, `yield`
物理	`set_gravity()`, `set_friction()`, `apply_force()`, `apply_impulse()`
音频	`AudioManager.load()`, `AudioManager.play()`, `play_music()`
动画	`AnimatedSprite2D`, `Tween`, `SpriteFrames`
摄像机	`Camera2D` 跟随/平滑
粒子	`ParticleEmitter2D` 各种预设
UI	`Label`, `Button`, `ProgressBar`, `Panel`
地图	`TileMap`, `TileSet`, `load_from_array()`
寻路	`NavigationAgent2D`, `NavigationGrid`
路径	`Path2D`, `PathFollow2D`, `Line2D`
资源	`ResourceLoader.load()`
信号	`Signal`, `.connect()`, `.emit()`
状态机	`StateMachine`, `State`

第八部分：项目文件格式

8.1 .scrawl 文件结构

项目文件是 JSON 格式，结构如下：

```json
{
    "name": "我的游戏",
    "version": "1.0",
    "game": {
        "width": 800,
        "height": 600,
        "title": "My Game",
        "fps": 60,
        "fullscreen": false,
        "debug": false
    },
    "scenes": [
        {
            "id": "uuid-string",
            "name": "MainScene",
            "background_color": [100, 150, 200],
            "background_image": null,
            "code": "class MainScene(Scene): ...",
            "sprites": [
                {
                    "id": "uuid-string",
                    "name": "Player",
                    "class_name": "Player",
                    "node_type": "Sprite",
                    "x": 400,
                    "y": 300,
                    "direction": 0,
                    "size": 1.0,
                    "visible": true,
                    "costumes": [],
                    "code": "class Player(Sprite): ...",
                    "is_physics": false,
                    "collision_type": "rect",
                    "properties": {}
                }
            ]
        }
    ],
    "scripts": {},
    "images": [],
    "sounds": []
}
```

8.2 项目目录结构

```
项目根目录/
├── 项目名.scrawl          # 主项目文件（JSON）
├── assets/
|   ├── images/             # 图片资源（PNG/JPG/GIF/BMP）
|   └── sounds/             # 音频资源（WAV/MP3/OGG）
└── scripts/                # 外部脚本文件（可选）
```

第九部分：键盘快捷键

快捷键	功能
`Ctrl+N`	新建项目
`Ctrl+O`	打开项目
`Ctrl+S`	保存项目
`Ctrl+Shift+S`	另存为
`Ctrl+Q`	退出 IDE
`Ctrl+Z`	撤销
`Ctrl+Y`	重做
`Ctrl+X`	剪切
`Ctrl+C`	复制
`Ctrl+V`	粘贴
`Ctrl+,`	打开设置
`F5`	运行游戏
`Shift+F5`	停止游戏

第十部分：已知限制与未完成功能

10.1 完成度总览

模块	状态	IDE 支持	引擎实现	运行时可用
Sprite	✅ 完成	✅ 完整	✅ 完整	✅ 可用
PhysicsSprite	✅ 完成	✅ 完整	✅ 完整	✅ 可用
AnimatedSprite2D	⚠️ 部分	✅ 有编辑器	⚠️ 逻辑完成	❌ 缺渲染
StaticBody2D	⚠️ 部分	✅ 属性完整	⚠️ 数据模型	❌ 无碰撞检测
RigidBody2D	⚠️ 部分	✅ 属性完整	⚠️ 有物理计算	❌ 无碰撞检测
KinematicBody2D	⚠️ 部分	✅ 属性完整	❌ 存根	❌ move_and_slide 不解决碰撞
Area2D	⚠️ 部分	✅ 属性完整	❌ 存根	❌ 检测不到重叠
Camera2D	✅ 完成	✅ 属性完整	✅ 完整	✅ 可用
PointLight2D	⚠️ 部分	✅ 属性完整	❌ 仅数据	❌ 无光照渲染

模块	状态	IDE 支持	引擎实现	运行时可用
DirectionalLight2D	⚠️ 部分	✅ 属性完整	❌ 仅数据	❌ 无光照渲染
Line2D	⚠️ 部分	✅ 有编辑器	⚠️ 数据完整	❌ 缺渲染
AudioPlayer2D	⚠️ 部分	✅ 属性完整	❌ 仅API	❌ 无实际播放
ParticleEmitter2D	✅ 完成	✅ 属性完整	✅ 完整	⚠️ 需渲染器
Path2D	✅ 完成	✅ 有编辑器	✅ 完整	✅ 逻辑可用
PathFollow2D	✅ 完成	✅ 有编辑器	✅ 完整	✅ 逻辑可用
TileMap	✅ 完成	✅ 有编辑器	✅ 完整	✅ 逻辑可用
Label	⚠️ 部分	✅ 属性完整	❌ 仅数据	❌ 无文字渲染
Button	⚠️ 部分	✅ 属性完整	⚠️ 信号可用	❌ 无 UI 渲染
ProgressBar	⚠️ 部分	✅ 属性完整	⚠️ 信号可用	❌ 无 UI 渲染
Panel	⚠️ 部分	✅ 属性完整	❌ 仅数据	❌ 无 UI 渲染
Timer	✅ 完成	✅ 属性完整	✅ 完整	✅ 可用
NavigationAgent2D	✅ 完成	✅ 有编辑器	✅ A*寻路完整	✅ 逻辑可用
StateMachine	✅ 完成	✅ 有片段	✅ 完整	✅ 可用
Tween	✅ 完成	✅ 有片段	✅ 完整	⚠️ 需手动update
信号系统	✅ 完成	—	✅ 完整	✅ 可用
输入系统	⚠️ 部分	—	✅ 逻辑完整	❌ 未接入引擎
数学库	✅ 完成	—	✅ 完整	✅ 可用

10.2 引擎层面未完成

🔴 严重（核心功能缺失）

1. 无纯 Python 游戏循环

- **问题**：`Game.run()` 完全依赖 Rust native bridge（`NativeGame`），没有 native bridge 时只打印提示信息就退出
- **影响**：如果没有编译 Rust 桥接库，游戏完全无法运行
- **需要**：实现一个基于 pygame 的纯 Python 渲染后端作为 fallback

2. 无碰撞检测系统

- **问题**：
  - `KinematicBody2D.move_and_slide()` 只移动位置，不检测/解决碰撞
  - `KinematicBody2D.is_on_floor/wall/ceiling()` 永远返回 `False`
  - `Area2D.get_overlapping_bodies()` 永远返回空列表
  - `RayCast2D.is_colliding()` 永远返回 `False`
  - `RigidBody2D` 有力学计算但不检测碰撞
- **影响**：所有物理节点的碰撞功能都不工作
- **需要**：实现 AABB / SAT 碰撞检测算法，碰撞事件触发

3. 无音频播放

- **问题**：`AudioPlayer.play()` 和 `AudioManager.play()` 只切换 `_playing` 标志，不产生声音
- **影响**：所有音频功能无效
- **需要**：集成 pygame.mixer 或其他音频库

4. 无 UI 渲染

- **问题**：Label、Button、ProgressBar、Panel 等 UI 控件只有数据模型，没有 `draw()` 方法
- **影响**：UI 控件在游戏中不可见
- **需要**：实现基于 pygame 的 UI 渲染层

🟡 中度（功能不完整）

5. AnimatedSprite2D 缺少图片加载

- **问题**：帧切换逻辑完整，但没有实际从磁盘加载和显示图片帧
- **需要**：接入渲染器的精灵帧加载

6. Light2D 无渲染

- **问题**：PointLight2D 和 DirectionalLight2D 只是数据对象，没有光照算法
- **需要**：实现 2D 光照混合（如 multiply blend + radial gradient）

7. Line2D / Path2D 无渲染

- **问题**：路径逻辑完整但不画线
- **需要**：接入渲染器绘制线段

8. Input 系统未接入引擎

- **问题**：`Input` 类逻辑完整，但引擎没有调用 `Input._on_key_press()` 等方法传递事件
- **需要**：在游戏循环中接入键盘/鼠标事件转发

9. Scene 不使用节点树

- **问题**：`Scene` 类使用简单列表管理精灵，没有继承 `Node`，不调用子节点的 `_process()`
- **需要**：Scene 应继承 Node，在游戏循环中递归调用场景树的 `_process` / `_physics_process`

10. Tween 需要手动更新

- **问题**：`Tween.update(delta)` 需要手动调用，没有自动集成到游戏循环
- **需要**：在 Node._process 中自动更新

11. ResourceLoader 不加载文件

- **问题**：`ResourceLoader.load()` 只创建包装对象，不真正读取文件内容
- **需要**：实现实际的图片/音频/字体文件加载

🟢 低度（可以后续完善）

12. compat.py 兼容层未实现

- **问题**：v1 兼容层完全空白
- **影响**：旧项目无法无缝迁移

13. 粒子系统需要渲染器

- **问题**：粒子逻辑完整（位置计算、生成销毁），但需要渲染器来绘制粒子
- **数据可用**：`get_particles()` 返回完整粒子列表

14. 容器布局只有基础实现

- **问题**：HBoxContainer 只做了简单的 x 偏移，VBoxContainer / GridContainer 更简单
- **需要**：完善布局算法（考虑 size flags、min_size 等）

10.3 IDE 层面未完成

🟡 中度

1. 撤销/重做系统

- **问题**：编辑菜单有撤销/重做选项但实际功能未接线（信号占位）
- **需要**：实现 QUndoStack 命令模式

2. 场景编辑器交互有限

- **问题**：场景视图有基本的节点视觉呈现，但拖拽交互可能不完善
- **需要**：完善节点的拖拽移动、多选、对齐辅助线

3. 节点层级（父子关系）

- **问题**：IDE 场景树显示为平铺列表（场景→精灵），不支持节点嵌套
- **需要**：支持将节点拖放到其他节点下形成父子层级

4. 动画编辑器缺少时间轴

- **问题**：只有帧动画编辑器（SpriteFrames），没有 AnimationPlayer 的关键帧时间轴编辑器
- **需要**：实现类似 Godot 的时间轴编辑器

🟢 低度

5. 代码编辑器功能

- **问题**：使用 QTextEdit/QPlainTextEdit，缺少高级代码编辑功能
- **可能改进**：
  - 语法高亮（可能有基础实现）
  - 自动补全
  - 错误标注
  - 行号
  - 折叠

6. 资源预览

- **问题**：资源浏览器显示文件列表，但缺少丰富的预览功能
- **需要**：图片缩略图、音频波形预览

7. 多场景编辑

- **问题**：可以添加多个场景，但场景间切换和管理的体验可能不够流畅

10.4 架构层面问题

1. 双重精灵体系

- `Sprite`（v1 兼容，使用命令队列）和 `Node2D`（Godot 风格，使用场景树）是两套并行架构
- 新的节点类型（Camera2D、Timer 等）继承 Node2D，与 Sprite 不在同一棵树上
- **建议**：统一到 Node2D 体系

2. 原生桥接依赖

- 引擎深度依赖 Rust NativeGame 桥接
- 纯 Python 层无法独立运行
- **建议**：实现 pygame 后端作为开发/测试用 fallback

3. Scene 与 Node 脱节

- `Scene` 使用列表管理 `Sprite`
- `Node` 使用树形结构管理子节点
- 两者没有统一接口
- **建议**：Scene 继承 Node，统一管理

> **总结**：IDE 的编辑功能（22 种节点类型、属性编辑、4 个专用可视化编辑器、代码生成、60+ 代码片段）相当完整。引擎的 **数据模型和逻辑层**（数学库、信号系统、状态机、寻路、粒子、路径、计时器、Camera2D）质量很高。主要短板在于 **引擎渲染层和碰撞系统未实现**——需要一个纯 Python 游戏循环后端（建议基于 pygame）来驱动所有已完成的逻辑代码。